

Task 3 — Titanic Survival Prediction

Notebook Link:

<https://colab.research.google.com/drive/1tAmq-cmjuCLnz32wA2biMKg0MuakZsrB?usp=sharing>

Summary:

Objective

The objective of this project is to build a machine learning model that predicts whether a passenger survived the Titanic disaster based on passenger details and travel information.

Dataset Used

The Titanic dataset containing features such as:

- Passenger Class
- Gender
- Age
- Fare
- Embarked Location
- Family Details
- Survival Status

Algorithms Used

- Random Forest Classifier

Steps Performed

- Imported and analyzed the dataset
- Checked and handled missing values
- Performed feature engineering
- Created new features such as Family Size and Cabin Presence
- Extracted passenger titles from names
- Encoded categorical variables
- Conducted Exploratory Data Analysis (EDA)
- Split dataset into training and testing sets
- Trained machine learning classification model
- Evaluated model performance
- Generated predictions on sample inputs
- Saved trained model using Joblib

Visualizations Included

- Survival Count Plot
- Correlation Heatmap

- Feature Importance Graph

Evaluation Metrics

- Accuracy Score
- Precision
- Recall
- F1-Score
- Confusion Matrix

Result

The Random Forest Classifier achieved good accuracy in predicting passenger survival. Feature engineering improved the model performance and helped identify important survival-related factors.

Tools and Technologies Used

- Python
- Pandas
- NumPy
- Matplotlib
- Seaborn
- Scikit-learn
- Google Colab

Screenshots:

+ <> + TT

Connect ^

v

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket
0	892	0	3	Kelly, Mr. James	male	34.5	0	0	330911
1	893	1	3	Wilkes, Mrs. James (Ellen Needs)	female	47.0	1	0	363272
2	894	0	2	Myles, Mr. Thomas Francis	male	62.0	0	0	240276
3	895	0	3	Wirz, Mr. Albert	male	27.0	0	0	315154
4	896	1	3	Hirvonen, Mrs. Alexander (Helga E Lindqvist)	female	22.0	1	1	3101298 1

[]

df.info()

v

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 418 entries, 0 to 417
Data columns (total 12 columns):
#   Column      Non-Null Count  Dtype
---  -
0   PassengerId  418 non-null    int64
1   Survived     418 non-null    int64
2   Pclass       418 non-null    int64
3   Name         418 non-null    object
4   Sex          418 non-null    object
5   Age          332 non-null    float64
6   SibSp        418 non-null    int64
7   Parch        418 non-null    int64
8   Ticket       418 non-null    object
9   Fare         417 non-null    float64
10  Cabin        91 non-null     object
11  Embarked     418 non-null    object
dtypes: float64(2), int64(5), object(5)
memory usage: 39.3+ KB
```

[]

df.isnull().sum()

v

	0
PassengerId	0
Survived	0
Pclass	0
Name	0
Sex	0
Age	86
SibSp	0
Parch	0
Ticket	0

+ <> + T

Connect ^

	PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket
0	892	0	3	Kelly, Mr. James	male	34.5	0	0	330911
1	893	1	3	Wilkes, Mrs. James (Ellen Needs)	female	47.0	1	0	363272
2	894	0	2	Myles, Mr. Thomas Francis	male	62.0	0	0	240276
3	895	0	3	Wirz, Mr. Albert	male	27.0	0	0	315154
4	896	1	3	Hirvonen, Mrs. Alexander (Helga E Lindqvist)	female	22.0	1	1	3101298 1

```
[ ] df['Age'] = df['Age'].fillna(df['Age'].mean())
df['Embarked'] = df['Embarked'].fillna(df['Embarked'].mode()[0])
print("Missing values handled successfully")
```

Missing values handled successfully

```
[ ] df.drop(['PassengerId', 'Name', 'Ticket', 'Cabin'], axis=1, inplace=True)
df.head()
```

	Survived	Pclass	Sex	Age	SibSp	Parch	Fare	Embarked	FamilySize	Ca
0	0	3	male	34.5	0	0	7.8292	Q	0	
1	1	3	female	47.0	1	0	7.0000	S	1	
2	0	2	male	62.0	0	0	9.6875	Q	0	
3	0	3	male	27.0	0	0	8.6625	S	0	
4	1	3	female	22.0	1	1	12.2875	S	2	

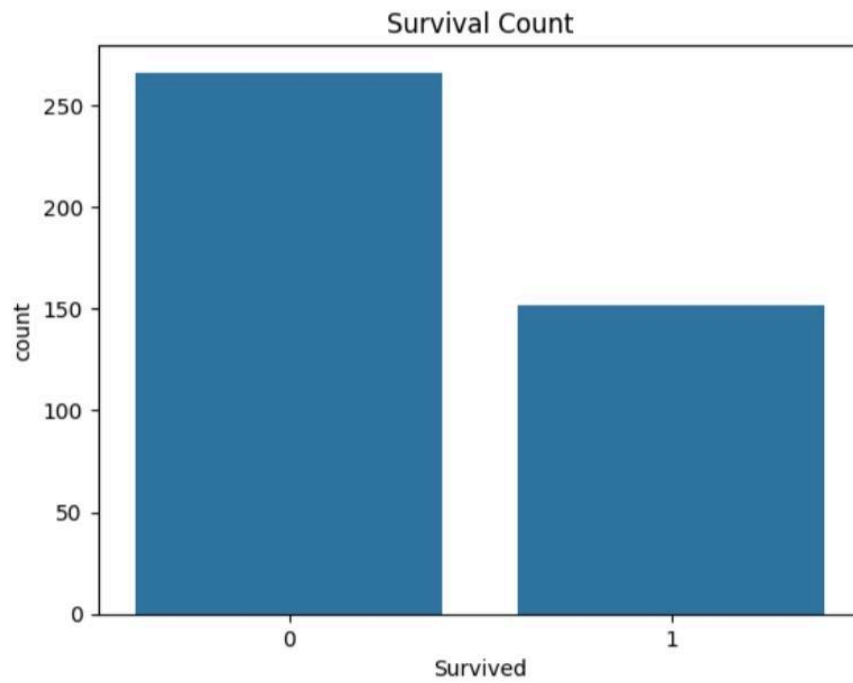
```
[ ] le = LabelEncoder()
for col in df.columns:
    if df[col].dtype == 'object':
        df[col] = le.fit_transform(df[col])
df.head()
```

	Survived	Pclass	Sex	Age	SibSp	Parch	Fare	Embarked	FamilySize	Cabi
0	0	3	1	34.5	0	0	7.8292	1	0	
1	1	3	0	47.0	1	0	7.0000	2	1	
2	0	2	1	62.0	0	0	9.6875	1	0	
3	0	3	1	27.0	0	0	8.6625	2	0	
4	1	3	0	22.0	1	1	12.2875	2	2	

+ <> + T

Connect ^

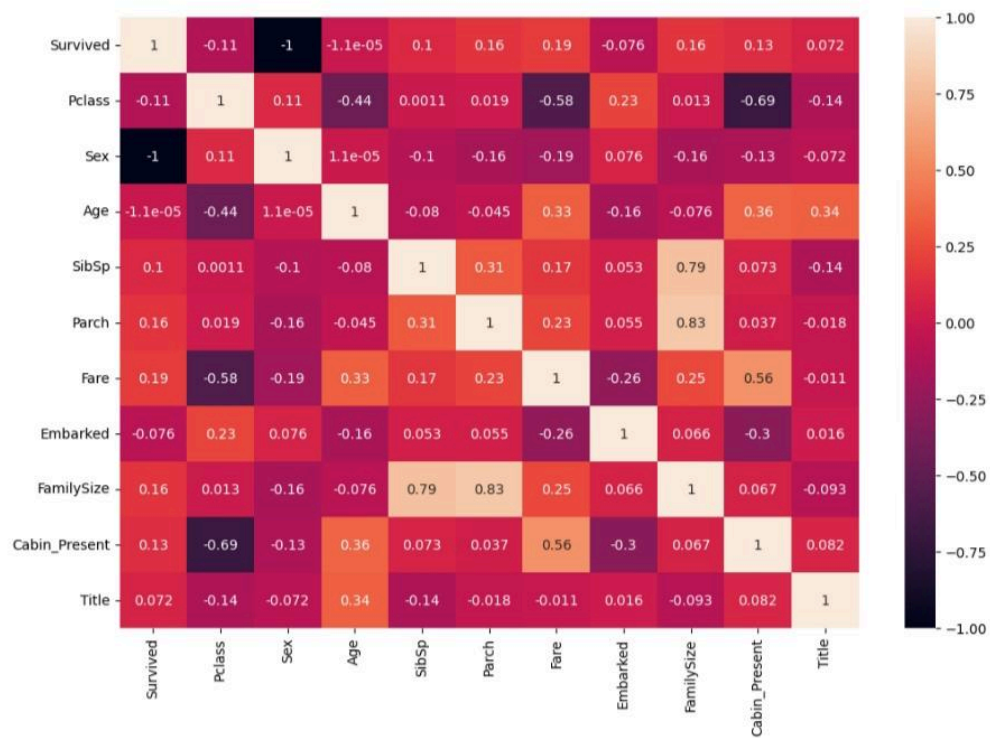
v



[]

```
plt.figure(figsize=(12,8))  
sns.heatmap(df.corr(), annot=True)  
plt.show()
```

v



+ <> + T

Connect ^

```
[ ] model.fit(X_train, y_train)

pred = model.predict(X_test)
```

```
[ ] acc = accuracy_score(y_test, pred)

print("Accuracy:", acc)
```

Accuracy: 1.0

```
[ ] print(classification_report(y_test, pred))
```

precision recall f1-score support

0	1.00	1.00	1.00	50
1	1.00	1.00	1.00	34
accuracy			1.00	84
macro avg	1.00	1.00	1.00	84
weighted avg	1.00	1.00	1.00	84

```
[ ] cm = confusion_matrix(y_test, pred)

print(cm)
```

[[50 0]
[0 34]]

```
[ ] importance = pd.Series(
    model.feature_importances_,
    index=X.columns
)

importance.sort_values().plot(
    kind='barh',
    figsize=(8,6)
)

plt.title("Feature Importance")

plt.show()
```

